

Documentation du projet BD

Naomie Kouassi , Léandre Javelot, Célestin Martin, Mattéo Bect

Table des matières

1	Analyse du sujet	2
1.1	Tableau de contraintes (DF, de valeur, de multiplicité)	2
1.2	Contraintes contextuelles	3
1.3	Justification des choix de modélisation	3
2	Conception Entités/Associations	4
2.1	Diagramme	4
2.2	Contraintes	4
3	Passage au relationnel	5
3.1	Traduction en relationnel du diagramme E/A	5
3.2	Analyse de Normalisation de la Base de Données	6
3.2.1	Première Forme Normale (1FN)	6
3.2.2	Deuxième Forme Normale (2FN)	7
3.2.3	Troisième Forme Normale (3FN)	7
3.2.4	Troisième Forme Normale de Boyce-Codd-Kent (3FNBCK)	7
4	Analyse des fonctionnalités	8
4.1	Fonctionnalité 1 : Passage d'une commande par un client	8
4.2	Fonctionnalité 2 : Alertes de péremption et ajustement de prix	8
4.3	Fonctionnalité 3 : Clôture d'une commande	8
4.4	Remarques concernant les procédures	9
5	Bilan du projet	10
5.1	Organisation	10
5.2	Points difficiles rencontrés	10
6	Mode d'emploi du démonstrateur	11
6.1	Aperçu de l'Interface Graphique	11
6.1.1	Connexion et Authentification	11
6.1.2	Catalogue et Panier (Vue Client)	11
6.1.3	Tableau de Bord Admin (Vue Épiciers)	11
6.2	Implémentation des Fonctionnalités	12
6.2.1	Gestion des Commandes (Transactionnelle)	13
6.2.2	Gestion des Stocks et Alertes (J-7)	13
6.2.3	Cycle de Vie et Clôture	13
6.2.4	Gestion des Pertes	13

1 Analyse du sujet

1.1 Tableau de contraintes (DF, de valeur, de multiplicité)

Dépendances Fonctionnelles	Contraintes Valeur	Contraintes Multiplicité
idProducteur $\rightarrow$ NomProducteur, NumProducteur, EmailProducteur, idAdresse idAdresse $\rightarrow$ Voie, CodePostal, Ville, Pays idActivite $\rightarrow$ NomActivite idProduit $\rightarrow$ NomProduit, Categorie, Description, Caractéristique, idProducteur idArticle $\rightarrow$ TypeArticle, idProduit idLot $\rightarrow$ DateRéception, DatePéremption, TypePéremption, QuantitéLot, PrixAchat, PrixVente, idArticle idDispo $\rightarrow$ MoisDebut, MoisFin idPerte $\rightarrow$ NaturePerte, DatePerte, QuantitéPerdue, idLot emailClient $\rightarrow$ NomClient, PrénomClient, NumClient, idClient idCommande $\rightarrow$ DateCommande, DateRecuperation, Statut, ModePaiement, ModeRécupération, PrixTTC, idClient idLigne $\rightarrow$ QuantitéCommandée, PrixUnitaire, Sous-Total, idCommande, idArticle	Latitude $\in [-90, 90]$ Longitude $\in [-180, 180]$ MoisDébut, MoisFin $\in \{1, \dots, 12\}$ MoisDébut $\leq$ MoisFin TypeArticle $\in \{\text{Vrac, PreCond, Contenant}\}$ Réutilisable $\in \{\text{TRUE, FALSE}\}$ Capacite > 0 PoidsSachet > 0 PrixAchat, PrixVente > 0 PrixVente $>$ PrixAchat DatePéremption $>$ DateRéception TypePéremption $\in \{\text{DLC, DLUO}\}$ QuantitéLot > 0 Statut $\in \{\text{En préparation, Prête, Récupérée, En livraison, Livrée, Annulée}\}$ ModePaiement $\in \{\text{EnLigne, EnBoutique}\}$ ModeRécupération $\in \{\text{Retrait, Livraison}\}$ DateLivraisonEstimée $>$ DateCommande PrixTTC > 0 QuantitéCommandée > 0 PrixUnitaire > 0 SousTotal > 0 FraisLivraison ≥ 0	idProducteur $\rightarrow$ idActivite, idProduit idAdresse $\nrightarrow$ idProducteur, idClient, FraisLivraison idActivite $\rightarrow$ idProducteur idProduit $\rightarrow$ idArticle idProduit $\nrightarrow$ idDispo idDispo $\rightarrow$ idProduit idArticle $\rightarrow$ idLot idArticle $\nrightarrow$ idLigne idLot $\nrightarrow$ idPerte idClient $\rightarrow$ idAdresse, idCommande idClient $\nrightarrow$ emailClient idCommande $\rightarrow$ idLigne idArticle $\nrightarrow$ Capacite, Reutilisable, PoidsSachet idAdresse $\nrightarrow$ Latitude, Longitude idCommande $\nrightarrow$ FraisLivraison, idAdresse

1.2 Contraintes contextuelles

Les articles peuvent être de type *Préconditionné*, *Vrac* ou *Contenant*. Ces ensembles sont deux à deux disjoints et leur réunion forme l'ensemble des articles :

$$\text{Ext}(\text{ArticlePréconditionné}) \cup \text{Ext}(\text{ArticleVrac}) \cup \text{Ext}(\text{ArticleContenant}) = \text{Ext}(\text{Article}),$$

$$\text{Ext}(\text{ArticlePréconditionné}) \cap \text{Ext}(\text{ArticleContenant}) = \emptyset,$$

$$\text{Ext}(\text{ArticlePréconditionné}) \cap \text{Ext}(\text{ArticleVrac}) = \emptyset,$$

$$\text{Ext}(\text{ArticleVrac}) \cap \text{Ext}(\text{ArticleContenant}) = \emptyset.$$

Une adresse peut être du type *AdresseProducteur*, c'est-à-dire comporter une géolocalisation :

$$\text{AdresseProducteur} \subset \text{Adresse}.$$

Une commande peut être du type *CommandeLivrée*; dans ce cas, elle possède des frais de livraison et une adresse associée :

$$\text{CommandeLivrée} \subset \text{Commande}.$$

1.3 Justification des choix de modélisation

Gestion des modes de conditionnement

Nous avons introduit une table *Article* afin de distinguer les différentes formes sous lesquelles un produit peut être vendu. Un article correspond à un produit, et possède un attribut *TypeArticle* permettant d'identifier son mode de conditionnement (Vrac, Préconditionné ou Contenant). Nous avons choisi de modéliser les contenants comme un type d'article au même titre que les autres modes de conditionnement, car ils partagent de nombreuses caractéristiques communes : provenance d'un producteur, association à des lots, prix, périodes de disponibilité, etc. Ce choix nous permet également d'introduire des sous-tables spécifiques lorsque certains types d'articles nécessitent des attributs supplémentaires (par exemple la capacité ou la réutilisabilité).

Saisonnalité

Un produit peut être disponible durant plusieurs périodes au cours d'une année, ou aucune. Pour représenter cette saisonnalité, nous avons créé une table *Disponibilité*, dans laquelle chaque période est identifiée par un *idDispo* et peut être associée à plusieurs produits. Nous avons ajouté la contrainte de cohérence *MoisFin* $\leq$ *MoisDébut*. Dans le cas d'une disponibilité qui s'étend sur la fin et le début d'année (par exemple de novembre à février), la période est découpée en deux intervalles distincts : (11, 12) et (1, 2). Ce choix permet de modéliser chaque année indépendamment des autres.

Gestion des pertes

Une perte est enregistrée au niveau du lot auquel elle se rapporte. Ainsi, si un incident touche plusieurs lots (par exemple un dégât des eaux), une perte est créée pour chacun d'eux. Cette modélisation assure une traçabilité précise des quantités perdues en affectant une quantité perdue par lot.

Droit à l'oubli

Les clients sont identifiés par un *idClient* interne, indépendant de leurs données personnelles. Cet identifiant peut être associé à un email afin de retrouver le nom, le prénom et le numéro du client. En cas de demande d'effacement des données personnelles, ces dernières peuvent être supprimées tandis que l'identifiant interne et l'historique des commandes sont conservés sous forme anonymisée.

Gestion des dates de péremption

Comme un lot ne peut posséder qu'un seul type de péremption, nous avons retenu deux attributs : *TypePéremption* (DLC ou DLUO) et *DatePéremption*. Ce modèle garantit l'exclusion mutuelle entre les deux types de dates.

Calcul des frais de livraison

En accord avec le client, nous avons retenu un modèle fondé sur le pays de destination (France, DOM-TOM ou International) et le poids total de la commande. Les frais sont fixés à 5 pour la France, 20 pour les DOM-TOM et 15 pour l'international, auxquels s'ajoute un supplément de 0,5 par kilogramme. Ce modèle a été choisi en s'inspirant des pratiques courantes.

2 Conception Entités/Associations

2.1 Diagramme

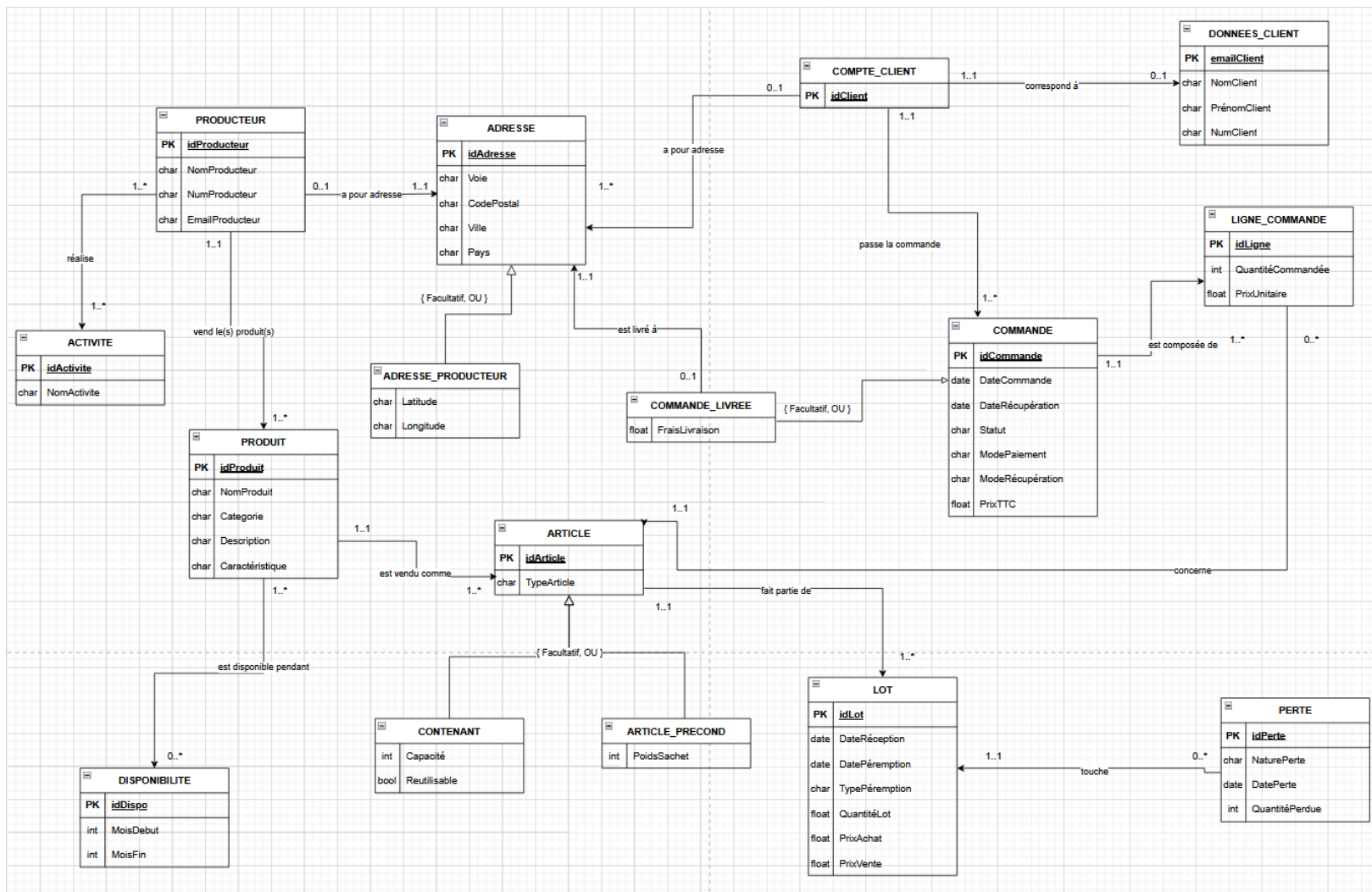


Figure 1: Diagramme Entité-Association

2.2 Contraintes

Toutes les contraintes de valeurs présentes dans le tableau de l'analyse font parties des contraintes qui ne sont pas représentées dans le schéma.

Il y a également quelques contraintes de contextes :

- Un produit de conditionnement vrac doit être vendu comme ARTICLE
- Un produit pré-conditionné comme ARTICLE_PRECOND
- Un contenant comme CONTENANT
- Un producteur doit avoir une adresse de type ADRESSE_PRODUCTEUR
- Une commande avec un ModeRécupération = Livraison, doit être de type COMMANDE_LIVREE

3 Passage au relationnel

3.1 Traduction en relationnel du diagramme E/A

Adresse(idAdresse, Voie, CodePostal, Ville, Pays)

Producteur(idProducteur, NomProducteur, NumProducteur, EmailProducteur, **idAdresse**)

- IdAdresse non nul et référence Adresse(idAdresse)

Afin d'éviter de laisser les colonnes Latitude et Longitude nulles pour un non-producteur, le sous-type Adresse_Producteur est représenté par référence :

Adresse_Producteur(**idAdresse**, Latitude, Longitude)

- idAdresse référence Adresse(idAdresse)

Activité(idActivité, NomActivité)

Puisque les producteurs peuvent avoir une ou plusieurs activités, on traduit cette association de cardinalité 1.. par la création d'une table Producteur_Activité qui fait le lien entre les deux :*

Producteur_Activité(idProducteur, **idActivité**)

- Toute activité doit être exercée par au moins un producteur
- Tout producteur doit exercer au moins une activité

Produit(idProduit, NomProduit, Catégorie, Description, Caractéristique, **idProducteur**)

- idProducteur non nul et référence Producteur(idProducteur)
- Tout producteur doit produire au moins un produit

Disponibilité(idDispo, MoisDebut, MoisFin)

*La table Produit_Dispos fait le lien entre un produit et ses éventuelles disponibilités pour traiter l'association de cardinalité 0..**

Produits_Dispos(**idProduit**, **idDispo**)

- idProduit référence Produit(idProduit)
- idDispo référence Disponibilité(idDispo)
- Toute disponibilité doit correspondre à au moins un produit

Le 'sur-type' Article donne les deux sous-types Contenant et Contenant. Ces sous-types sont représentés par référence et non par duplication afin de garder le lien avec la table Article.

Nous avons fait le choix de regrouper les contenants, article en vrac et article pré-conditionnés dans une seule et même table afin de faciliter les manipulations dans les transactions notamment. Si on recherche l'article auquel correspond un lot (par exemple pour la gestion des dates de péremption), on n'a pas de disjonction de cas à faire selon le type d'article.

Article(idArticle, TypeArticle, **idProduit**)

- idProduit non nul et référence Produit(idProduit)
- Tout produit doit donner au moins un article

Contenant(**idArticle**, Capacité, Réutilisable)

- idArticle référence Article(idArticle)

Article_Precond(idArticle, PoidsSachet)

- idArticle référence Article(idArticle)

Lot(idLot, **idArticle**, DateReception, DatePeremption, TypePeremption, QuantiteLot, PrixAchat, PrixVente, PromoAppliquee)

- idArticle non nul et référence Article(idArticle)

Perte(idPerte, NaturePerte, DatePerte, QuantitéPerdue, **idLot**)

- idLot non nul et référence Lot(idLot)

Compte_Client(idClient)

DonnéesClient(emailClient, NomClient, PrénomClient, NumClient, idClient)

- idClient non nul et référence Compte_Client(idClient)

Un client pouvant avoir plusieurs adresses, Adresses_Pour_Clients fait le lien entre la table Adresse et l'idClient :

Adresses_Pour_Clients(idAdresse, **idClient**)

- idAdresse référence Adresse(idAdresse)
- IdClient référence idClient (Compte_Client)

Commande(idCommande, **idClient**, DateCommande, DateRécupération, Statut, ModePaiement, ModeRécupération, PrixTTC)

- IdClient non nul et référence idClient (Compte_Client)

Afin d'éviter la duplication de valeurs, le sous-type commande livrée est représenté par référence :

Commande_Livrée(idCommande, FraisLivraison, **idAdresse**)

- idCommande référence Commande(idCommande)
- idAdresse non nul et référence Adresse(idAdresse)

Ligne_Commande(idLigne, QuantitéCommandée, PrixUnitaire, SousTotal, idArticle, **idCommande**)

- idArticle non nul et référence Article(idArticle)
- idCommande non nul et référence Commande(idCommande)

3.2 Analyse de Normalisation de la Base de Données

3.2.1 Première Forme Normale (1FN)

Règle :

- Tous les attributs doivent être atomiques et non Null (une seule valeur par cellule).

Vérification :

- Les tables comme PRODUIT, PRODUITEUR, ADRESSE, COMMANDE ont toutes des colonnes atomiques.

Conclusion : La 1FN est respectée.

3.2.2 Deuxième Forme Normale (2FN)

Règle :

- La table doit être en 1FN.
- Tous les attributs non clefs sont pleinement dépendants de chacune des clefs.

Vérification :

- Toutes les tables ont une clé primaire simple (`idProduit`, `idClient`, `idCommande`, etc.), donc les dépendances partielles sont impossibles.

Conclusion : La 2FN est respectée.

3.2.3 Troisième Forme Normale (3FN)

Règle :

- La table doit être en 2FN.
- Tout attribut non clef ne dépend pas d'un ensemble d'attributs qui n'est pas une clef.

Vérification :

On utilise la définition équivalente : une relation R (et ses DF F) est en troisième forme normale (3FN) si et seulement si tout attribut non clef est pleinement et directement dépendant de toutes les clefs.

- En observant attentivement chaque table, tout attribut non clef est pleinement et directement dépendant de toutes les clefs si on supprime la ligne Sous-Total de la ligne de commande car cette quantité dépend de la quantité commandée et du prix unitaire au moment de l'achat : nous avons donc supprimé cet élément du schéma EA tout en le conservant en SQL pour le calcul du PrixTTC.

Conclusion : La 3FN est respectée suite à cette modification.

3.2.4 Troisième Forme Normale de Boyce-Codd-Kent (3FNBCK)

Règle :

- La table doit être en 3FN.
- Pour toute DF $(X \twoheadrightarrow Y)$ non triviale (i.e. $Y \subsetneq X$), X contient une clef de R .

Vérification :

- Toutes les dépendances fonctionnelles impliquent une clé unique à gauche (qui est donc une clé de R).

Conclusion : La 3FNBCK est respectée.

4 Analyse des fonctionnalités

4.1 Fonctionnalité 1 : Passage d'une commande par un client

Le passage d'une commande par un client implique les actions suivantes :

- Vérification du stock disponible
 - Vérifier que les quantités demandées sont disponibles.
 - Gérer le prélèvement selon la stratégie FIFO (lots triés par date de péremption).
 - Détecter un stock insuffisant avant de créer la commande.
- Vérification de la saisonnalité
 - Si aucune période n'est associée, il est considéré disponible toute l'année.
 - Sinon, on vérifie si le mois actuel appartient à une période autorisée.
 - Si ce n'est pas le cas, le système doit indiquer la prochaine période disponible.
- Calcul du montant total
 - Calculer le prix de chaque ligne : quantité $\times$ prix unitaire.
 - Additionner les sous-totaux (et éventuellement les frais de livraison) pour obtenir le PrixTTC.
- Gestion des frais de livraison
 - S'ils s'appliquent, les frais sont ajoutés au total et enregistrés dans *COMMANDE_LIVREE*.
- Création propre de la commande
 - Insérer un en-tête dans la table *COMMANDE*.
 - Insérer les lignes dans *LIGNE_COMMANDE*.
 - Finaliser le prix total.
- Gestion des erreurs
 - Stock insuffisant $\rightarrow$ message explicite.
 - Produit hors saison $\rightarrow$ prochaine disponibilité.

4.2 Fonctionnalité 2 : Alertes de péremption et ajustement de prix

Cette transaction doit :

- Identifier les lots périmant dans les 7 prochains jours.
- Proposer une réduction automatique des prix. Nous avons choisi, comme suggéré dans l'énoncé de réduire de 30% le prix. On a dû ajouter un "flag" pour ne pas que la réduction se fasse à chaque fois que le gérant lance l'analyse.
- Fournir une liste lisible des produits qui vont être prochainement périmés, pour les gérants (résultat de l'analyse).
- Mettre à jour le prix dans la base.

4.3 Fonctionnalité 3 : Clôture d'une commande

À la fin d'une commande (retrait / livraison), il faut :

- Mettre à jour le statut de la commande en fonction du mode de récupération.
- En cas de paiement en boutique $\rightarrow$ enregistrer le paiement.
- En cas de livraison $\rightarrow$ les frais de livraison ont déjà été traités.
- Enregistrer la date de clôture.
- Empêcher la clôture si le statut n'est pas valide.

4.4 Remarques concernant les procédures

Nous avons appris lors de la soutenance que l'utilisation des procédures était fortement déconseillée. Cependant, il était trop tard pour modifier l'ensemble de nos transactions. Nous avons dans un premier temps commencé à utiliser le niveau **serializable** à la place des procédures. Face aux problèmes causés par son utilisation, nous sommes finalement passés à l'usage des procédures. Cependant, si nous avions maintenu l'usage de **serializable**, par exemple pour la transaction 2, les différences auraient été les suivantes :

Utilisation de Serializable

Listing 1: Extrait de code avec **serializable**

```
— Définir le niveau d'isolement de la transaction
EXECUTE IMMEDIATE 'SET TRANSACTION ISOLATION LEVEL SERIALIZABLE';

— Boucle pour traiter les lots avec date de peremption proche
FOR r IN (
    SELECT l.idLot, p.NomProduit, a.TypeArticle, l.DatePeremption, l.PrixVente
    FROM LOT l
    JOIN ARTICLE a ON l.idArticle = a.idArticle
    JOIN PRODUIT p ON a.idProduit = p.idProduit
    WHERE l.DatePeremption <= SYSDATE + 7
    AND l.DatePeremption > SYSDATE
    AND l.PromoAppliquee = 'N'
    AND l.QuantiteLot > 0
) LOOP
    — Mise a jour du prix du lot avec un rabais
    UPDATE LOT
    SET PrixVente = PrixVente * 0.7,
        PromoAppliquee = 'Y'
    WHERE idLot = r.idLot;

    COMMIT; — Validation de la transaction

    — Pipeline pour retourner les informations sur les lots modifies
    PIPE ROW(TLIGNE.PROMO(
        r.idLot,
        r.NomProduit,
        r.TypeArticle,
        r.DatePeremption,
        r.PrixVente,
        r.PrixVente * 0.7
    ));
END LOOP;
```

Dans cet extrait, nous voyons que le niveau d'isolement est défini au début de la transaction à l'aide de la commande **SET TRANSACTION ISOLATION LEVEL SERIALIZABLE**. Chaque lot sélectionné est mis à jour, et les modifications sont validées immédiatement avec **COMMIT**. Cependant, cela pourrait entraîner des blocages si d'autres transactions concurrentes tentent de modifier les mêmes lots.

Utilisation des procédures

En revanche, l'usage des procédures permet d'encapsuler la logique de mise à jour dans un bloc autonome. Une procédure peut être utilisée pour effectuer plusieurs mises à jour sans impacter l'isolement des transactions globales, ce qui peut améliorer la gestion des performances. Voici un extrait du code avec une procédure :

Listing 2: Extrait de code avec procédures

```
CREATE OR REPLACE PROCEDURE TRAITER_PEREMPTIONS_AUTO (
    p_resultat OUT SYS.REFCURSOR
) AS
BEGIN
    — Ouverture d'un curseur pour recuperer les lots a traiter
    OPEN p_resultat FOR
```

```

SELECT v.idLot ,
       v.NomProduit ,
       v.NomArticle AS TypeArticle ,
       v.DatePeremption ,
       v.Prix_Vente_Lot AS Ancien_Prix ,
       (v.Prix_Vente_Lot * 0.7) AS Nouveau_Prix
FROM V_DETAILS_LOTS v
JOIN LOT l ON v.idLot = l.idLot
WHERE v.DatePeremption <= SYSDATE + 7
AND v.DatePeremption > SYSDATE
AND l.PromoAppliquee = 'N';

```

— *Mise à jour du prix de vente dans la table LOT*

```

UPDATE LOT
SET PrixVente = PrixVente * 0.7 ,
    PromoAppliquee = 'Y'
WHERE DatePeremption <= SYSDATE + 7
AND DatePeremption > SYSDATE
AND PromoAppliquee = 'N'
AND QuantiteLot > 0;

```

COMMIT; — *Validation de la transaction*

END;

Dans cet extrait, la procédure `TRAITER_PEREMPTIONS_AUTO` permet de traiter en une seule opération l'ensemble des lots qui doivent être mis à jour. Elle utilise un curseur pour sélectionner les lots à traiter, et applique les modifications directement dans la base de données. Contrairement à l'approche avec `serializable`, les transactions sont gérées de manière autonome, ce qui simplifie la logique de mise à jour mais peut rendre l'application plus difficile à maintenir si des erreurs surviennent.

5 Bilan du projet

5.1 Organisation

Dès le début du projet, nous nous sommes répartis les tâches : en sachant que nous n'étions que quatre alors que le projet est initialement prévu pour cinq personnes, nous savions qu'il fallait être le plus efficace possible pour terminer le projet dans les temps. Nous avons donc attribué à Léandre le développement de l'interface en Java dès le premier jour, il a donc été le candidat sur la partie EA (puisque'il n'a pas travaillé dessus). Naomie, Mattéo et Célestin ont d'abord travaillé ensemble sur l'analyse du problème sous forme de contraintes puis sur la conception Entités/Associations car ces parties étaient essentielles pour l'ensemble du projet : s'accorder à trois au début était donc la meilleure solution. Ensuite, nous nous sommes répartis les trois fonctionnalités tout en demandant à Léandre comment son interface en Java était conçue pour être en accord avec cette interface. Nous avons fait pour cela des réunions hebdomadaires pour discuter de l'avancée du projet. La semaine avant le rendu, nous avons écrit cette documentation du projet, chacun étant responsable d'une partie du document.

5.2 Points difficiles rencontrés

Les premières difficultés furent liées au diagramme Entités/Associations où certaines parties ont pu nous porter à confusion, nécessitant quelques appels au corps enseignant pour obtenir des réponses à nos questionnements. Ensuite, les fonctionnalités et transactions nous ont posé quelques difficultés par leur complexité à l'implémentation. A posteriori, il semble aussi que confier à Léandre la partie Java dès la première séance n'était peut-être pas la meilleure idée : la qualité de l'interface n'étant pas évaluée, nous aurions probablement dû nous concentrer dans un premier temps sur le diagramme EA et les dépendances fonctionnelles avant de commencer la partie Java, beaucoup moins importante dans le projet. Au cours de cette partie Java, Léandre a lui aussi rencontré quelques problèmes car le module JDBC lui était inconnu au début du projet mais la lecture de la documentation lui a permis de surmonter ces difficultés.

6 Mode d'emploi du démonstrateur

6.1 Aperçu de l'Interface Graphique

Cette section illustre le parcours utilisateur à travers les différents écrans de l'application.

6.1.1 Connexion et Authentification

L'application démarre par une fenêtre de connexion sécurisée. Le système détecte automatiquement le rôle de l'utilisateur (Client ou Épicier) en fonction de l'email saisi. Saisir "admin@valaisan.ch" pour l'accès administrateur, et "jean.dupont@mail.com" pour un exemple d'accès client.

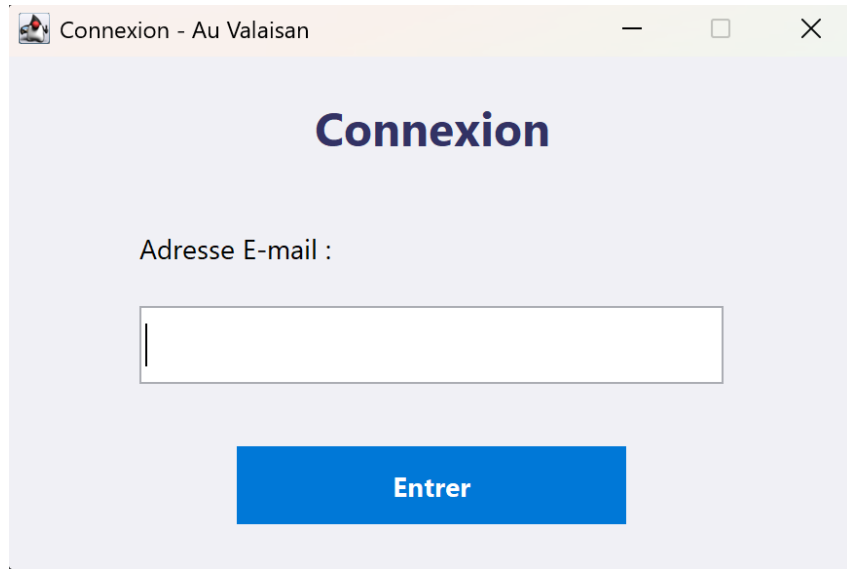


Figure 2: Fenêtre d'authentification

6.1.2 Catalogue et Panier (Vue Client)

Le client accède au catalogue global. Les produits hors saison sont automatiquement filtrés ou signalés. Le panier permet d'agréger plusieurs articles avant la validation.

6.1.3 Tableau de Bord Admin (Vue Épicier)

L'épicier dispose d'onglets supplémentaires pour gérer les stocks critiques, valider les préparations et gérer les pertes.

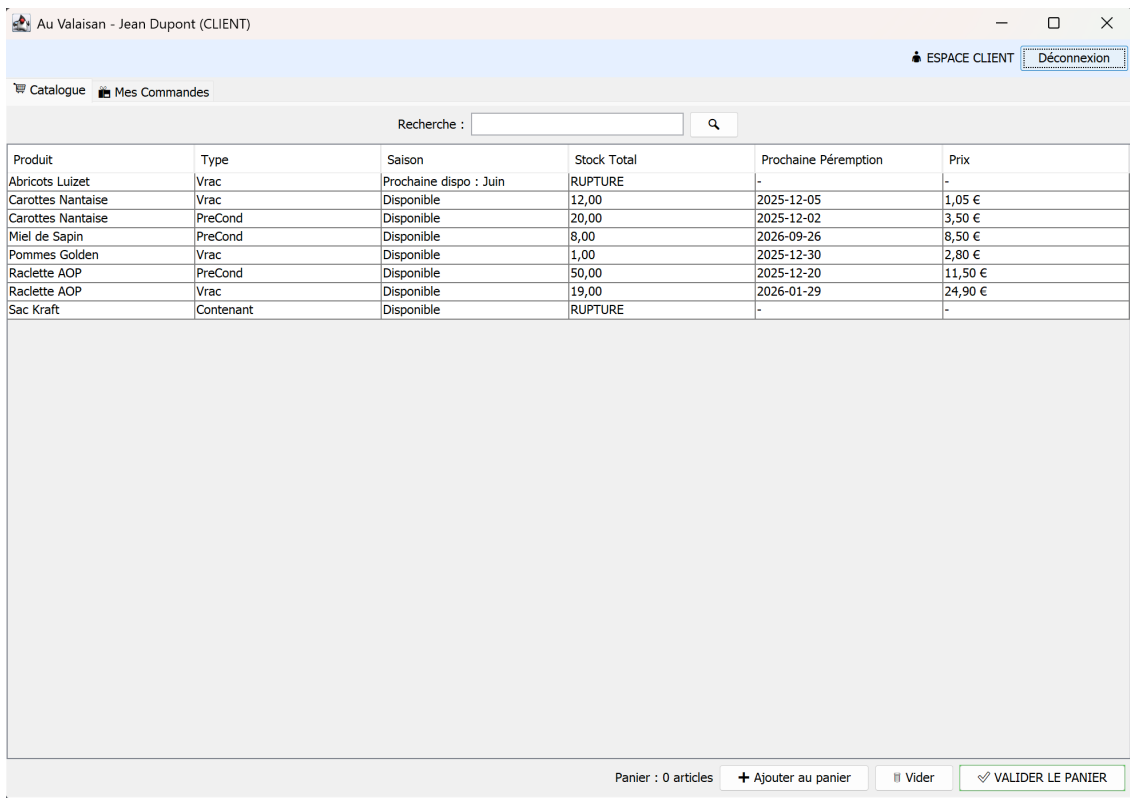


Figure 3: Interface de commande avec gestion du panier

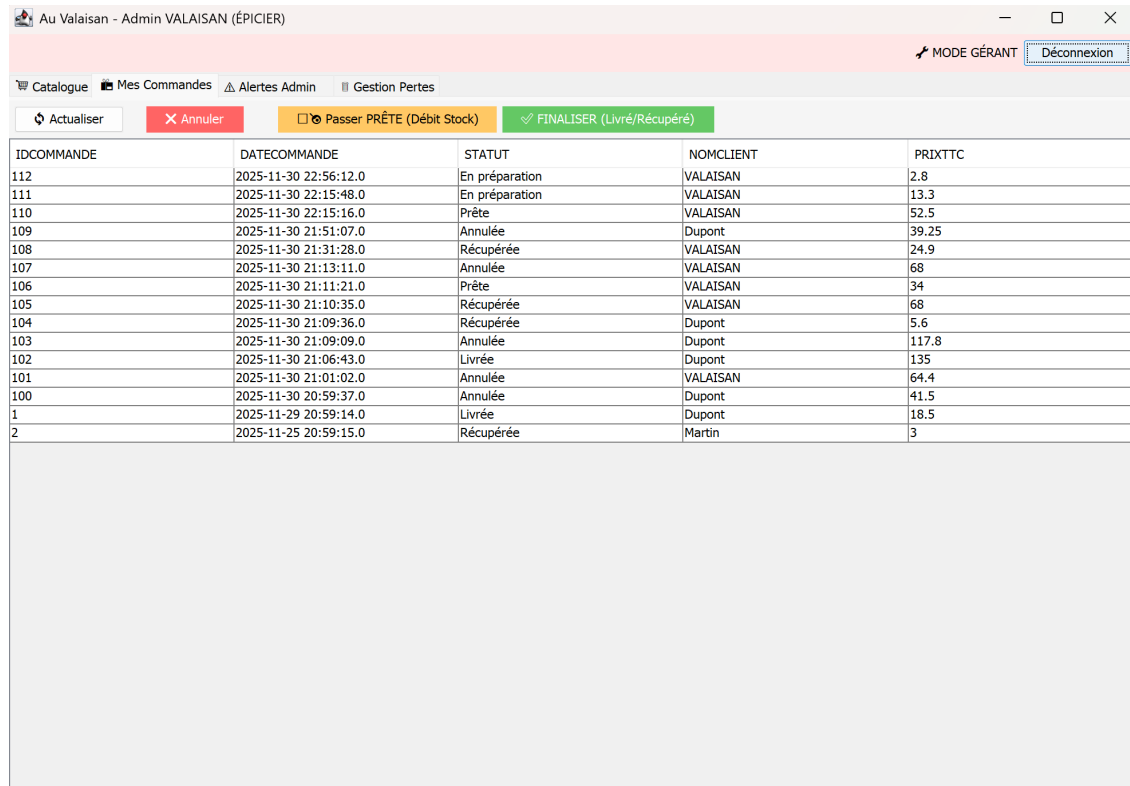


Figure 4: Interface de suivi et de clôture des commandes

6.2 Implémentation des Fonctionnalités

Il y a une délégation forte de la manipulation de la base de données au SGBD Oracle, pour limiter son accès en Java.

6.2.1 Gestion des Commandes (Transactionnelle)

La fonctionnalité de commande assure qu'aucun stock n'est promis s'il n'est pas disponible.

- **Saisonnalité** : Une fonction SQL `GET_INFO_SAISON` vérifie en temps réel si le produit est disponible le mois courant.
- **Vérification FIFO** : La procédure `PASSER_COMMANDE` appelle la fonction `CALCULER_PANIER` qui détermine quels lots précis (les plus anciens) seront affectés à la commande.
- **Gestion des Unités** : Le système gère la conversion automatique entre la demande client (en kg) et le stock réel (unités de sachets ou vrac).
- **Intégrité** : L'ensemble du processus (création commande, lignes, calcul prix) est encapsulé dans une transaction unique.

6.2.2 Gestion des Stocks et Alertes (J-7)

Une fonctionnalité d'alerte automatisée a été développée.

- **Détection** : Une procédure stockée scanne quotidiennement les lots dont la date de péremption est dans moins de 7 jours.
- **Action automatique** : Les prix de ces lots sont réduits de 30% et un indicateur `PromoAppliquee` est mis à jour pour éviter une double réduction.
- **Visualisation** : L'interface Admin permet de lancer cette analyse et de visualiser les impacts sur les prix des lots.

6.2.3 Cycle de Vie et Clôture

Le cycle de vie d'une commande est stricte :

1. **En préparation** : Le stock est réservé "virtuellement" mais physiquement présent.
2. **Prête** : L'épicier valide la préparation. C'est à cet instant précis que la procédure `PREPARER_COMMANDE` débite physiquement les lots (`DELETE` si quantité = 0).
3. **Livrée / Récupérée** : La procédure `CLOTURER_COMMANDE` finalise la transaction, calcule les frais de livraison finaux (selon le pays et le code postal) et archive la date de fin.

6.2.4 Gestion des Pertes

L'administrateur peut déclarer des pertes (Casse, Vol) directement liées à un lot spécifique. Une contrainte d'intégrité vérifie que la perte déclarée ne dépasse pas le stock disponible.